

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

на создание информационной системы учёта заявок на грузоперевозки
«ТРАНСЛОГ»

1. Общие положения

1.1. Наименование системы

Автоматизированная информационная система учёта заявок на грузоперевозки транспортного предприятия (далее — Система). Условное название: «ТРАНСЛОГ».

1.2. Цель создания

Автоматизация процессов приёма, обработки, планирования, выполнения и учёта заявок на грузоперевозки автомобильным транспортом, повышение прозрачности работы предприятия, сокращение трудозатрат персонала и улучшение качества обслуживания клиентов.

1.3. Задачи системы

- приём заявок на перевозку от клиентов и отслеживание их статусов в режиме онлайн;
- обработка заявок менеджерами: смена статуса, назначение транспортного средства и водителя;
- автоматический расчёт стоимости рейса;
- учёт транспортного парка, типов транспортных средств и водителей;
- ведение справочников (типы ТС и тарифы, роли, статусы заявок);
- разграничение прав доступа для клиентов, менеджеров, водителей и администраторов;
- формирование выходной информации: списки заявок и рейсов, квитанция об оплате, сводная статистика.

1.4. Термины и сокращения

- ТС — транспортное средство;
- заявка — запрос клиента на перевозку груза из пункта отправления в пункт прибытия;
- рейс — выполнение заявки назначенным ТС и водителем;

- тариф — базовая стоимость за 1 км пути, заданная для типа ТС;
- API — программный интерфейс серверной части (Web API);
- EF Core — Entity Framework Core, ORM для работы с БД;
- СУБД — система управления базами данных.

2. Назначение и цели создания системы

2.1. Назначение

Система предназначена для автоматизации работы транспортного предприятия, оказывающего услуги по перевозке грузов автомобильным транспортом, и обеспечивает работу клиентов, менеджеров, водителей и администраторов в едином информационном пространстве.

2.2. Целевые показатели эффективности

- отказ от бумажных документов при приёме и учёте заявок: 100% заявок оформляются в Системе;
- исключение ошибок ручного расчёта: стоимость рейса определяется автоматически;
- время отклика API на типовые операции — не более 2 секунд;
- назначение ТС и водителя на заявку выполняется менеджером в одной форме;
- клиент в любой момент видит актуальный статус и историю своих заявок.

3. Характеристика объекта автоматизации

Объект автоматизации — транспортное предприятие, оказывающее услуги по перевозке грузов автомобильным транспортом.

- пользователи: клиенты, менеджеры, водители, администраторы;
- расчётная нагрузка (для проектирования): до 50 одновременных пользователей, до 100 ТС, до 1000 заявок в месяц;
- ключевые правила предметной области: каждый водитель закреплён за одним ТС и выполняет рейсы только на нём; стоимость рейса зависит от длины маршрута и типа ТС.

4. Требования к системе

4.1. Требования к системе в целом

Система строится по клиент-серверной архитектуре с разделением ответственности:

- серверная часть — API на C# (ASP.NET Core), слои: модели, контроллеры, контекст БД (EF Core);
- клиентская часть — веб-приложение на JavaScript (Vue.js, сборщик Vite);
- хранилище — централизованная БД под управлением MySQL 8.0.

Система должна быть модульной: компоненты клиентской части и контроллеры API выделяются по ролям и сущностям.

4.2. Требования к функциональным подсистемам

4.2.1. Подсистема «Авторизация и учётные записи»

- регистрация пользователя: логин, пароль, полное имя, номер телефона;
- аутентификация по логину и паролю; определение роли и переход на главную страницу, соответствующую роли;
- уникальность логина; учётная запись со статусом «неактивна» не может войти в Систему.

4.2.2. Подсистема «Клиент»

- подача заявки: пункт отправления, пункт прибытия, вес груза (т), объём груза (м³), описание груза (необязательно);
- просмотр статусов активных заявок;
- история заявок со сводной статистикой: всего, в работе, доставлено, в ожидании;
- отмена заявки — только в статусах «ожидание» и «принято»;
- оплата заявки в статусе «ожидает оплаты»; получение квитанции;
- доступ только к собственным заявкам; подача заявки только после авторизации.

4.2.3. Подсистема «Менеджер»

- просмотр списка всех заявок со сводной статистикой;

- изменение статуса заявки;
- назначение ТС и водителя на заявку: выбор ТС, дата отправления, дата прибытия, длина маршрута (км);
- просмотр списка водителей: ФИО, водительские категории, закреплённое ТС, грузоподъёмность.

4.2.4. Подсистема «Водитель»

- просмотр назначенных рейсов: маршрут, даты, расстояние, вес груза, контакты клиента;
- установка статуса рейса — только «выполняется» или «доставлено»;
- доступ только к собственным рейсам и только на закреплённом ТС.

4.2.5. Подсистема «Администрирование и справочники»

- управление пользователями: добавление, удаление, изменение данных, назначение одной роли из допустимых (клиент, менеджер, водитель, администратор), изменение статуса активности;
- управление транспортным парком: добавление, редактирование, удаление ТС (госномер, марка, модель, грузоподъёмность в кг, объём кузова в м³, тип ТС, водитель — необязательно);
- справочники: типы ТС с тарифом за 1 км (редактируемый), просмотр ролей и статусов заявок.

4.2.6. Жизненный цикл заявки

Статусы заявки: «ожидание», «принято», «ожидает оплаты», «выполняется», «доставлено», «отменено».

- после назначения ТС и расчёта стоимости заявка получает статус «ожидает оплаты»;
- статусы «выполняется» и «доставлено» отмечает водитель; остальные статусы меняет менеджер;
- статус «отменено» устанавливается при отмене заявки клиентом.

4.2.7. Выходная информация

- список заявок клиента: дата, маршрут, вес груза, статус, стоимость;

- список всех заявок для менеджера: номер, дата поступления, маршрут, вес груза, статус;
- список рейсов водителя: маршрут, даты, расстояние, вес груза, контакты клиента;
- квитанция об оплате: номер заявки, маршрут, расстояние, тип ТС, стоимость за 1 км, итоговая стоимость;
- сводная статистика по заявкам.

4.3. Требования к интеграции

- взаимодействие клиента и сервера — через Web API по HTTP в формате JSON (контроллеры: Auth, Order, Vehicle, Driver, VehicleType, User, Role);
- интеграции с внешними системами (платёжные шлюзы, картографические сервисы, бухгалтерия) в рамки данного проекта не входят и рассматриваются как возможное развитие.

5. Требования к видам обеспечения

5.1. Математическое и алгоритмическое обеспечение

- стоимость рейса = длина маршрута (км) × базовая стоимость за 1 км типа ТС;
- длина маршрута и тариф — строго положительные числа;
- вес груза не должен превышать грузоподъёмность назначаемого ТС;
- дата прибытия не может быть раньше даты отправления;
- ТС назначается только при наличии закреплённого за ним водителя.

5.2. Информационное обеспечение

БД проектируется в MySQL 8.0 (MySQL Workbench) и включает таблицы:

- roles — роли пользователей;
- users — пользователи (связь с roles);
- vehicle_types — типы ТС и тариф за 1 км;
- vehicles — ТС (связь с vehicle_types, уникальный госномер);
- drivers — водители (связь 1:1 с users и связь с vehicles);
- orders — заявки (связь с users и vehicles).

История заявок хранится в БД без ограничения срока.

5.3. Программное обеспечение

- серверная часть: C#, ASP.NET Core Web API, Entity Framework Core, СУБД MySQL 8.0;
- клиентская часть: Vue.js, Vite; для запуска и сборки — Node.js версии 22.16.0 и выше;
- клиентские ОС: Windows 10 и выше, Linux, macOS; современный веб-браузер;
- среда разработки: Visual Studio Code, MySQL Workbench.

5.4. Техническое обеспечение

Минимальные требования к клиентскому компьютеру:

- ОЗУ — не менее 2 ГБ;
- процессор Intel Pentium 2,16 ГГц и выше;
- свободное место на диске — не менее 1 ГБ;
- доступ в интернет — не менее 1 Мбит/с;
- периферия: мышь, клавиатура, монитор.

Рекомендуемые параметры сервера: 2 ядра CPU, 4 ГБ ОЗУ, SSD от 40 ГБ.

5.5. Эргономика и интерфейс

- отдельный интерфейс для каждой роли, навигация через боковое меню;
- пользователю достаточно базовых навыков работы с ПК;
- валидация форм с понятными сообщениями («Заполните все поля», «Заказ добавлен!»);
- минимальное число действий для типовых операций: подача заявки, назначение ТС, смена статуса.

6. Требования к надёжности и безопасности

- разграничение прав доступа: клиент видит только свои заявки, водитель — только свои рейсы, администрирование доступно только роли «администратор»; проверка выполняется на стороне сервера;
- сохранность данных: возможность создания и восстановления резервной копии БД;

- защита учётных данных: пароли хранятся в виде хэша, а не открытым текстом; в эксплуатации обмен данными — по HTTPS;
- устойчивость к ошибкам пользователя: проверка данных на клиенте и на сервере; отказы программы из-за некорректных действий недопустимы;
- блокировка входа для пользователей со статусом «неактивен».

7. Этапы разработки и внедрения

Таблица 1 – Этапы разработки и внедрения

Этап	Содержание работ	Результат
1. Анализ и проектирование	Изучение существующих решений, описание предметной области, диаграммы прецедентов и развёртывания, проектирование БД	ТЗ, UML-диаграммы, схема БД
2. Серверная часть	Создание БД, API на ASP.NET Core, аутентификация, расчёт стоимости	Рабочий API
3. Клиентская часть	Интерфейсы клиента, менеджера, водителя и администратора на Vue.js	Веб-приложение
4. Тестирование	Проверка на корректных и некорректных данных, контрольный пример	Протокол тестирования
5. Документирование	Подготовка пояснительной записки и руководства пользователя	Комплект документации
6. Внедрение	Развёртывание, наполнение справочников, обучение пользователей	Акт ввода в эксплуатацию

8. Критерии приёмки

Система считается принятой, если:

- выполнены все функциональные требования п. 4 для всех четырёх ролей;
- тесты (авторизация с верными и неверными данными, подача заявки, назначение ТС, валидация формы) пройдены без ошибок;

- стоимость рейса на контрольном примере совпадает с расчётом по формуле п. 5.1;
- нарушающие ограничения операции отклоняются (вес больше грузоподъёмности, дата прибытия раньше отправления, недопустимый статус);
- персонал выполняет базовые операции по руководству пользователя без ошибок;
- подписан акт ввода в эксплуатацию.

9. Требования к документированию

В комплект поставки входят:

- пояснительная записка;
- руководство пользователя (клиент, менеджер, водитель, администратор);
- инструкция по установке и запуску (БД, API, клиентское приложение: `npm i`, `npm run dev`);
- описание API (контроллеры и методы);
- схема БД и UML-диаграммы.

Согласовано:

Заказчик: _____ / _____

Исполнитель: _____ / _____